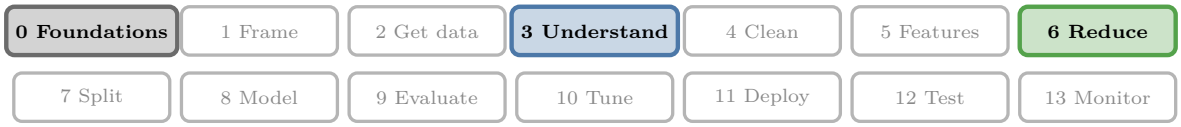
PCA: Problem Formulation and Step-by-Step Solution

Contents

1. Overview . . . 2
2. The problem PCA solves . . . 2
   - 2.1 Projecting one point . . . 3
   - 2.2 The variance of all the projections . . . 4
   - 2.3 The objective . . . 5
3. Covariance and the covariance matrix . . . 6
   - 3.1 What variance cannot see . . . 6
   - 3.2 Covariance . . . 7
   - 3.3 The covariance matrix . . . 8
4. Eigenvectors and eigenvalues . . . 9
   - 4.1 A matrix is a transformation . . . 9
   - 4.2 The vectors that do not turn . . . 9
   - 4.3 The eigenvectors of the covariance matrix . . . 11
5. PCA in five steps . . . 13
   - 5.1 The projection step . . . 14
6. PCA by hand in Python . . . 15
   - 6.1 Reading the result: the recipe and the scree plot . . . 17
7. Summary . . . 20
8. Sources . . . 20
9. Key terms . . . 21

Where this fits: Pipeline map steps 0 (Foundations), 3 (Understand data), 6 (Reduce dimensions). See the Course map.



- **Builds on:** Dot product; Covariance and covariance matrix; Linear transformations and matrices; Eigenvectors and eigenvalues; PCA.
- **Compare with:** Correlation; Singular value decomposition; Low-rank approximation (truncated SVD).

1. Overview

Key point: PCA looks for the **unit vector** (G-2048) (a direction) along which the projected data has the largest variance (spread). The answer is the top eigenvector of the data's covariance matrix (a table of how the features vary and move together; both terms are built in sections 3 and 4).

PCA by its geometry built the idea geometrically: turn a line through the data, and keep the direction where the shadows spread the most. This Note turns that idea into mathematics and then into five concrete steps.

In plain words, PCA does three things:

1. shift the data so its centre sits at the origin;
2. find the line through the origin along which the points' shadows spread out the most;
3. describe every point by where its shadow falls on that line.

Figure 7 in Section 5 shows these steps as a picture; Sections 2 to 4 build the tools behind them.

Many ML algorithms are, underneath, a mathematical problem with a goal to optimise, called the **objective function** (G-1372). This Note:

1. writes PCA's objective function (section 2);
2. learns the two tools its solution uses: the **covariance matrix** (G-495) (section 3) and eigenvectors (section 4);
3. runs the five steps on real numbers (sections 5 and 6).

2. The problem PCA solves

Key point: Drop every point's shadow onto a line; PCA wants the line whose shadows are spread out the most.

Picture the flats of feature extraction (each flat is a dot, with its number of rooms and washrooms as the two coordinates) lit from the side, so that every dot casts a shadow on a line. A line along the cloud gives widely spread shadows; a line across the cloud squeezes

them together. The **objective function** (G-1372) of PCA is to find the line with the widest spread of shadows. Sections 2.1 to 2.3 turn this into numbers: first one shadow, then the spread of many shadows, then the objective.

2.1 Projecting one point

> **Key point:** The shadow of a point x on a line with direction u is a single number, found by a dot product.

Take data with two **features** (G-772) (input variables, the columns of the data table); each data point is one **observation** (G-1374) (one row). Each point has an x and a y value, so we can treat it as a **vector** (G-2081), an arrow from the origin to the point.

We want to project the point onto some line through the origin. Only the line's direction matters, not its length, so we describe it by a **unit vector** (G-2048) u: a vector of length 1 pointing along the line. Figure 1 shows a point x, a unit vector u, and the shadow of x on the line.

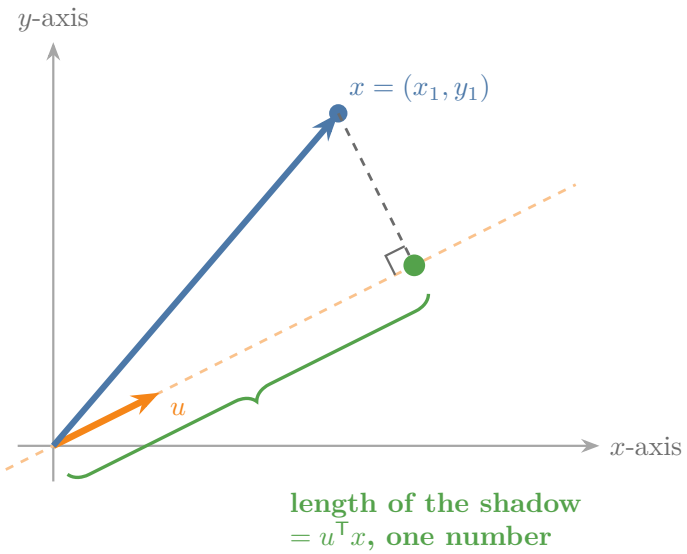


Figure 1: Projecting a point onto a unit vector

In words: the length of the shadow is the **dot product** (G-634) of u and x (multiply matching components, then add the products), divided by the length of u. Since u has length 1, the division disappears. The compact way to write the dot product is $u^\mathsf{T}x$: the **transpose** (G-2012) u^T is u written as a row, and a row times a column is the dot product.

$$\text{shadow length} = \frac{u \cdot x}{\|u\|} = u^\mathsf{T}x$$

With numbers: for $u = (0.894, 0.447)$, a unit vector, and the point $x = (2.4, 2.8)$:

$$u^\mathsf{T}x = 0.894 \times 2.4 + 0.447 \times 2.8$$

$$= 2.146 + 1.252$$

$$= 3.40$$

The point is now described by one number, 3.40, its position along the line. Two features have become one.

> **Extra:** u^T is u written as a row instead of a column (its transpose). A row times a column is the dot product, so $u^\mathsf{T}x$ is just a compact way to write $u \cdot x$.

2.2 The variance of all the projections

> **Key point:** Projecting all n points gives n numbers; their variance tells us how much of the spread the direction u keeps.

We project every point the same way and get one number per point. Here n is the number of points, and x_i is the i-th point, for example $x_2 = (0, 0)$ below. We try it on the three points of Data A (Section 3.1), $(-1, -1)$, $(0, 0)$ and $(1, 1)$, which already have mean $(0, 0)$.

For the line $u = (0.707, 0.707)$ (along the points):

point	$u^\mathsf{T}x$
$(-1, -1)$	$0.707 \times (-1) + 0.707 \times (-1) = -1.414$
$(0, 0)$	0
$(1, 1)$	$0.707 \times 1 + 0.707 \times 1 = 1.414$

The mean of the three shadows is 0. Their variance, the average of the squared distances from that mean:

$$\frac{(-1.414)^2 + 0^2 + 1.414^2}{3}$$

$$= \frac{2 + 0 + 2}{3} = 1.33$$

For the line $u = (0.707, -0.707)$ (across the points):

point	$u^\mathsf{T}x$
$(-1, -1)$	$0.707 \times (-1) + (-0.707) \times (-1) = 0$

point	$u^\mathsf{T}x$
$(0, 0)$	0
$(1, 1)$	$0.707 \times 1 + (-0.707) \times 1 = 0$

All three shadows land on one spot, so the variance is 0. The line along the points keeps the spread (1.33); the line across loses it all (0). The general formula does the same for any n. The mean of the shadows is the shadow of the mean point, $u^\mathsf{T}\bar{x}$, and the variance is the average squared distance of each shadow from it:

$$\sigma^2(u) = \frac{1}{n}\sum_{i=1}^{n}\left(u^\mathsf{T}x_i - u^\mathsf{T}\bar{x}\right)^2$$

With the numbers above, $n = 3$ and the formula gives 1.33 and 0. For the flats data of why PCA maximises variance (rooms and washrooms, 30 flats), the same formula gave 1.33 for u along the rooms axis and 2.61 for u at 45°.

2.3 The objective

> **Key point:** PCA's objective: among all **unit vectors** (G-2048), find the u with the largest $\sigma^2(u)$.

Any unit vector could be the answer, and each one gives a different variance. PCA wants the one with the most. In symbols, among all u with length 1, pick the one that makes the variance as large as possible:

$$\max_{\|u\|=1} \sigma^2(u)$$

Figure 2 draws $\sigma^2(u)$ for every direction on the flats data. The curve is highest, 2.61, at 45 degrees, and lowest, 0.05, at right angles to it: the two principal components found by turning a line. Section 4 shows that the peak sits at the top eigenvector of the **covariance matrix** (G-495), and its height is the largest eigenvalue.

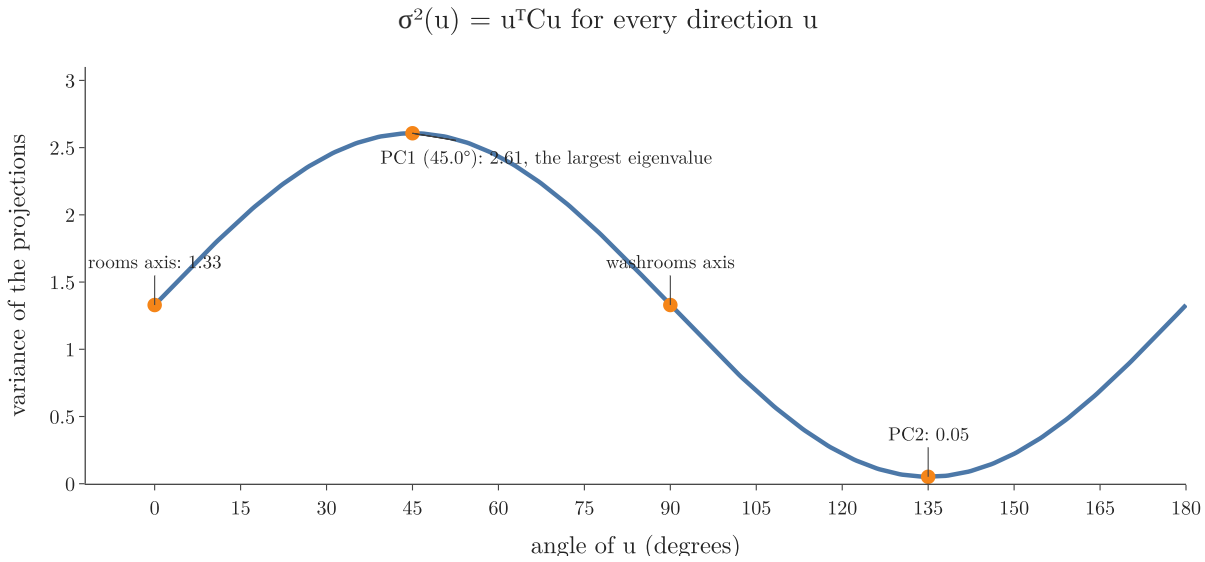


Figure 2: The variance of the projections of the 30 flats for every direction u, from 0 to 180 degrees. The maximum, 2.61 at 45 degrees, is PC1; the minimum, 0.05 at 135 degrees, is PC2.

This maximisation is PCA's **objective function** (G-1372). Solving the maximisation needs optimisation methods covered later; here we jump to the answer. To understand the answer, we need two ideas first: covariance and eigenvectors.

> **Extra:** The standard derivation rewrites $\sigma^2(u)$ as $u^\mathsf{T}Cu$, where C is the covariance matrix of Section 3. Maximising $u^\mathsf{T}Cu$ under the condition $u^\mathsf{T}u = 1$ (with a method called Lagrange multipliers) gives exactly $Cu = \lambda u$: the **eigenvector** (G-666) equation of Section 4 (Bishop, §12.1.1).

3. Covariance and the covariance matrix

> **Key point:** Variance describes the spread along one feature. Covariance describes how two features move together. The **covariance matrix** (G-495) holds both for every feature and every pair.

3.1 What variance cannot see

> **Key point:** Two datasets can have identical variances in every **feature** (G-772) and still point in opposite directions.

Variance is computed one feature at a time. Variance says how spread out x is and how spread out y is, but nothing about how x and y relate. Figure 3 shows two tiny datasets.

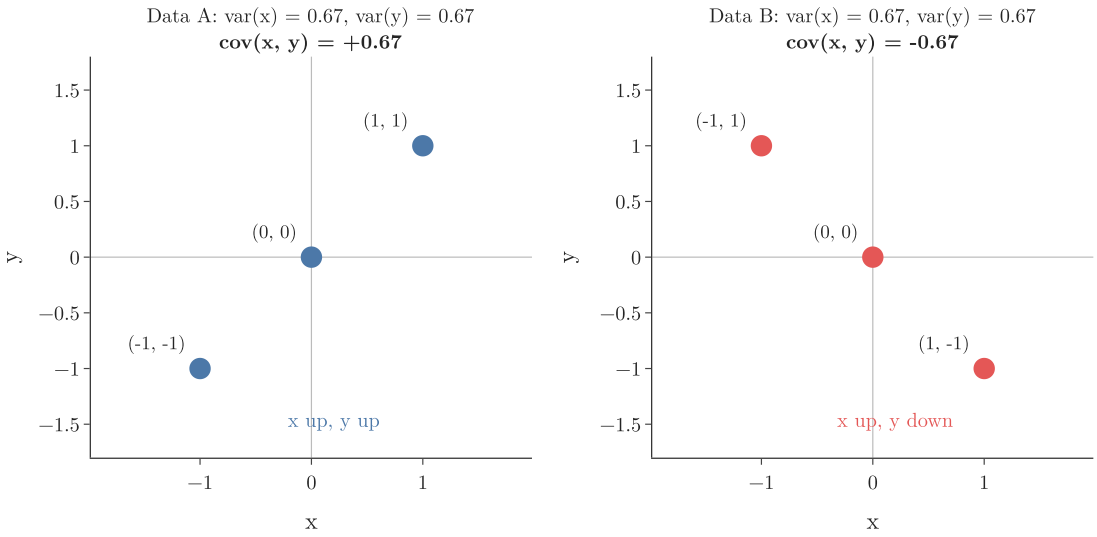


Figure 3: Same variances, opposite covariance

- **Data A:** $(-1, -1), (0, 0), (1, 1)$.
- **Data B:** $(-1, 1), (0, 0), (1, -1)$.

In both, x has variance 0.67 and y has variance 0.67. Yet in A the points rise from left to right, and in B they fall. Variance alone cannot tell them apart.

3.2 Covariance

> **Key point:** Covariance is positive when two **features** (G-772) rise together and negative when one rises as the other falls.

In words: for each point, multiply its distance from the mean in x by its distance from the mean in y, then average these products.

$$\text{cov}(x, y) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

With numbers: in both datasets the means are 0.

Data A:

$$\frac{(-1)(-1) + (0)(0) + (1)(1)}{3}$$

$$= \frac{2}{3} \approx +0.67$$

Data B:

$$\frac{(-1)(1) + (0)(0) + (1)(-1)}{3}$$

$$= -\frac{2}{3} \approx -0.67$$

The sign tells the direction of the relationship: positive in A, where x and y go up together, negative in B, where one goes up as the other goes down.

> **Extra:** Correlation (see correlation) is covariance divided by the two standard deviations. Dividing squeezes correlation into the range -1 to $+1$. Covariance has no fixed range: its size depends on the units of the columns.

3.3 The covariance matrix

> **Key point:** The covariance matrix puts each **feature** (G-772)'s variance on the diagonal and each pair's covariance off the diagonal. The matrix describes both the spread and the orientation of the data.

For data with features x, y and z, the **covariance matrix** (G-495) is the 3×3 table in Figure 4. With d features, it is $d \times d$.

	x	y	z
x	$\text{var}(x)$	$\text{cov}(x, y)$	$\text{cov}(x, z)$
y	$\text{cov}(y, x)$	$\text{var}(y)$	$\text{cov}(y, z)$
z	$\text{cov}(z, x)$	$\text{cov}(z, y)$	$\text{var}(z)$

diagonal: the variance of each column (the spread)

off the diagonal: the covariance of each pair (the orientation); same colour = same value, so the matrix is symmetric

Figure 4: The covariance matrix of three features

- **The diagonal** holds the variance of each feature. The covariance of a feature with itself is its variance: $\text{cov}(x, x) = \text{var}(x)$.
- **Off the diagonal** sit the covariances of each pair of features.
- **It is symmetric:** $\text{cov}(x, y) = \text{cov}(y, x)$, so the part above the diagonal mirrors the part below.

Variances plus covariances make the covariance matrix a complete summary for PCA. The diagonal gives the spread along each axis; the off-diagonal entries give the orientation of the cloud.

For the flats data (rooms and washrooms):

$$C = \begin{pmatrix} 1.33 & 1.28 \\ 1.28 & 1.33 \end{pmatrix}$$

Both features have variance 1.33, and their large positive covariance, 1.28, says they rise together. The computer (`np.cov` on the 30 flats, input the two columns, output this matrix) does the sums; each entry is the formula of Section 3.2 applied to one pair of features.

4. Eigenvectors and eigenvalues

> **Key point:** A matrix moves **vectors** (G-2081) around. Its eigenvectors are the special vectors it only stretches or shrinks, without turning them.

4.1 A matrix is a transformation

> **Key point:** Multiplying by a matrix moves every point of the plane: it can rotate, stretch, squash or shear the whole grid.

Think of a matrix as an action on the whole plane. Every point, seen as a **vector** (G-2081), is multiplied by the matrix and lands somewhere new. Straight grid lines stay straight and evenly spaced, but the grid can be turned, stretched or slanted. Such a change is called a **linear transformation** (G-1097).

The **identity matrix** (G-915) $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ leaves every vector where it was. Any other matrix moves vectors.

4.2 The vectors that do not turn

> **Key point:** An eigenvector keeps its direction under the transformation; its eigenvalue is how much it is stretched.

Figure 5 applies the matrix $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$ to the whole plane and follows three **vectors** (G-2081).

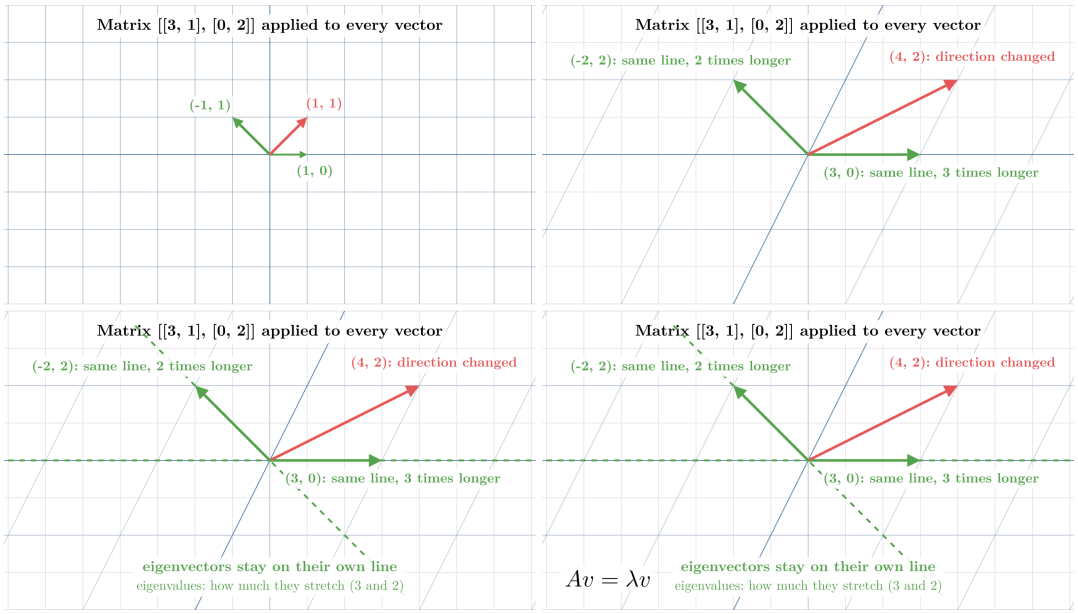


Figure 5: Applying a matrix: most vectors turn, eigenvectors only stretch. Idea and matrix after 3Blue1Brown, "Eigenvectors and eigenvalues".

1. $(1, 1)$ (red) becomes $(4, 2)$: its direction changes.
2. $(1, 0)$ (green) becomes $(3, 0)$: same line, 3 times longer.
3. $(-1, 1)$ (green) becomes $(-2, 2)$: same line, 2 times longer.

Think of pulling a rubber sheet sideways: arrows drawn on it mostly tilt, but an arrow drawn exactly along the pull just gets longer. Vectors that stay on their own line are the **eigenvectors** (G-666) of the matrix. The factor by which each one is stretched is its **eigenvalue** (G-665): 3 for $(1, 0)$ and 2 for $(-1, 1)$. A 2×2 matrix usually has two eigenvector directions, a 3×3 matrix three, and so on. Some have fewer (a rotation turns every vector, so it has none) and some have more (the identity matrix keeps every vector, so every direction is an eigenvector, with eigenvalue 1). A **covariance matrix** (G-495) always has at least the full number at right angles to each other (Section 4.3, Extra).

In words: applying the matrix to an eigenvector is the same as multiplying it by a plain number, its eigenvalue, written λ (lambda) here. Later Notes on ridge regression use λ for a penalty strength, which is a different quantity.

$$Av = \lambda v$$

With numbers, for $v = (-1, 1)$:

$$\begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$
$$= \begin{pmatrix} 3 \times (-1) + 1 \times 1 \\ 0 \times (-1) + 2 \times 1 \end{pmatrix}$$

$$= \begin{pmatrix} -2 \\ 2 \end{pmatrix}$$

$$= 2 \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

so $\lambda = 2$. An eigenvalue can also be negative (the vector flips to point the other way) or between 0 and 1 (it shrinks).

NumPy finds the eigenvectors for us (Section 6). Finding the eigenvectors shows how to find them by hand.

4.3 The eigenvectors of the covariance matrix

> **Key point:** The eigenvector of the **covariance matrix** (G-495) with the largest eigenvalue points along the greatest spread of the data. That eigenvector is PC1, and its eigenvalue is the variance along it.

The top **eigenvector** (G-666) of the covariance matrix solves PCA's objective from Section 2 (Bishop, §12.1.1). Use the covariance matrix as the transformation and find its eigenvectors:

- The eigenvector with the **largest eigenvalue** points in the direction of greatest variance. That eigenvector is the first principal component.
- Its **eigenvalue** equals the variance of the data projected onto it.
- The next eigenvector (next largest **eigenvalue** (G-665)) is PC2, at right angles to PC1, and so on.

Figure 6 checks this on the flats data.

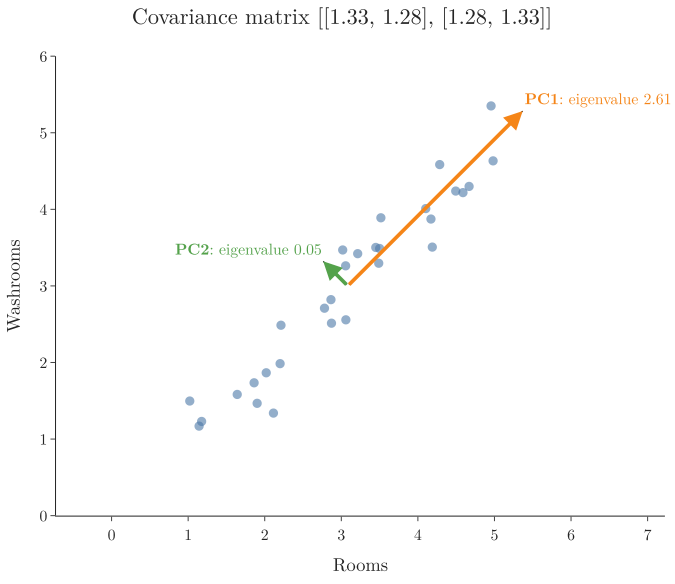


Figure 6: The eigenvectors of the covariance matrix of the flats data

The covariance matrix is:

$$\begin{pmatrix} 1.33 & 1.28 \\ 1.28 & 1.33 \end{pmatrix}$$

It has eigenvectors $(0.707, 0.707)$ and $(-0.707, 0.707)$, with eigenvalues 2.61 and 0.05. The computer finds them (Section 6); we can check them by hand with the equation $Av = \lambda v$ of Section 4.2, using the simpler lengths $(1, 1)$ and $(-1, 1)$, which lie on the same two lines:

Call the covariance matrix A. For $v = (1, 1)$:

$$A \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$
$$= \begin{pmatrix} 1.33 + 1.28 \\ 1.28 + 1.33 \end{pmatrix}$$
$$= \begin{pmatrix} 2.61 \\ 2.61 \end{pmatrix}$$
$$= 2.61 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

For $v = (-1, 1)$:

$$A \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$
$$= \begin{pmatrix} -1.33 + 1.28 \\ -1.28 + 1.33 \end{pmatrix}$$
$$= \begin{pmatrix} -0.05 \\ 0.05 \end{pmatrix}$$
$$= 0.05 \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

The eigenvalues are 2.61 and 0.05. Dividing $(1, 1)$ by its length 1.414 gives the **unit vector** (G-2048) $(0.707, 0.707)$.

The first eigenvector points at 45°, exactly the direction found by turning a line and measuring. Its eigenvalue, 2.61, is the variance we measured there. The second has eigenvalue 0.05, the small spread left at right angles.

So instead of trying every angle, PCA computes the eigenvectors of one matrix. The eigenvector method works the same way with 3, 10 or 1,000 **features** (G-772).

Extra: A covariance matrix is symmetric, and a symmetric $d \times d$ matrix always has d eigenvectors at right angles to each other (Strang, §6.4). Its eigenvalues are never negative, because each one is a variance: for an eigenvector u, $\lambda = u^{\mathsf{T}} C u$, the variance of the projections onto u (Section 2.3, Extra). These two facts make the principal components a proper new set of axes.

5. PCA in five steps

Key point: Centre the data, build the **covariance matrix** (G-495), find its eigenvectors, keep the top k, and project the data onto them.

Figure 7 summarises the whole method, with the shapes for data of 40 **observations** (G-1374) and 3 features reduced to 2.

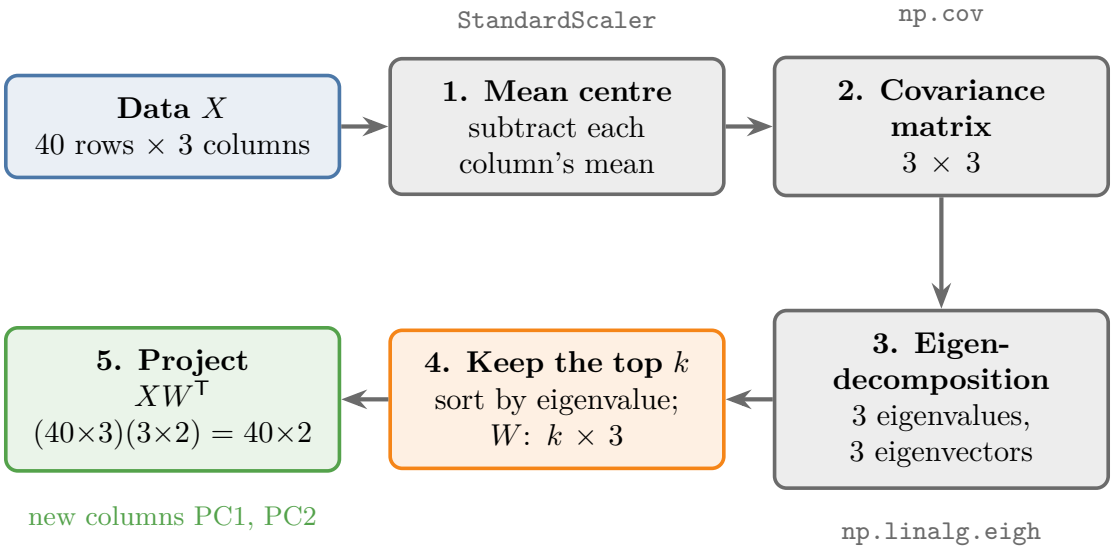


Figure 7: PCA step by step

1. **Mean centring** (G-1195). Subtract each **feature** (G-772)'s mean, so the data is centred on the origin.
2. **Covariance matrix.** Compute it from the centred data: 3×3 for 3 features.
3. **Eigen-decomposition** (G-663). Find the **eigenvalues** (G-665) and eigenvectors of the covariance matrix: 3 of each for 3 features.
4. **Keep the top k.** Sort the **eigenvectors** (G-666) by eigenvalue, largest first. Keep the first k as the rows of a matrix W ($k \times 3$). Choosing $k = 2$ goes from 3D to 2D; $k = 1$ goes to 1D.
5. **Project.** Multiply the data by W^{T}.

5.1 The projection step

> **Key point:** Projecting all points at once is one matrix multiplication: XW^T.

In words: each point's new coordinates are its **dot products** (G-634) with each kept eigenvector, as in Section 2.1. Doing this for all points at once is a matrix product.

$$Z = XW^\mathsf{T}$$

With shapes: X has 40 rows and 3 columns; W^T has 3 rows and 2 columns. The inner sizes (3 and 3) match, and the result Z has 40 rows and 2 columns: the new **features** (G-772) PC1 and PC2.

One row by hand. Use the two components of the 3-feature example of Section 6.1 as the rows of W, and an example centred **observation** (G-1374) $x = (1, 2, 0)$ (made up for this check):

$$W =$$

$$\begin{pmatrix} 0.54 & 0.66 & 0.53 \\ -0.69 & -0.01 & 0.72 \end{pmatrix}$$

$$\text{PC1} = 1 \times 0.54 + 2 \times 0.66 + 0 \times 0.53$$

$$\text{PC1} = 0.54 + 1.32 + 0 = 1.86$$

$$\text{PC2} = 1 \times (-0.69) + 2 \times (-0.01)$$

$$+0 \times 0.72$$

$$\text{PC2} = -0.69 - 0.02 + 0 = -0.71$$

So the three numbers $(1, 2, 0)$ become the two numbers $(1.86, -0.71)$: one row of Z. The matrix product repeats this for all 40 rows.

For 1D instead, W has one row, W^T is 3×1 and Z is 40×1. The **target** (G-1949) (the output we predict), if there is one, is copied across unchanged: PCA only transforms the inputs.

> **Extra: Mean centring** (G-1195) matters for the projection, not for the components. `np.cov` centres the data internally, so the **covariance matrix** (G-495) and the principal components are the same either way. The projection XW^T, though, only gives centred coordinates if X itself was centred. scikit-learn's `PCA` always centres for us.

6. PCA by hand in Python

> **Key point:** The five steps take a few lines of NumPy, and the result matches scikit-learn's PCA.

The example data has 40 points with 3 **features** (G-772), in two classes of 20. Each class is a cloud of points drawn at random around its own centre: $(0, 0, 0)$ for one class and $(1, 1, 1)$ for the other.

> **Python:** Steps 1 to 3.

```
from sklearn.preprocessing import StandardScaler

# step 1: centre (StandardScaler also scales each column)
X = StandardScaler().fit_transform(df[cols])
# step 2: covariance matrix; np.cov wants columns as rows
C = np.cov(X.T)
# step 3: eigh is made for symmetric matrices
values, vectors = np.linalg.eigh(C)
```

Step 1 here also divides each column by its standard deviation. PCA looks for large variance, so without this a feature measured in bigger numbers would take over PC1 because of its units alone (what standardising does).

The **covariance matrix** (G-495) is

$$C =$$

$$\begin{pmatrix} 1.026 & 0.205 & 0.080 \\ 0.205 & 1.026 & 0.198 \\ 0.080 & 0.198 & 1.026 \end{pmatrix}$$

All three features have about the same variance, and the covariances are small and positive. The computer (`np.linalg.eigh`, input C, output the **eigenvalues** (G-665) and eigenvectors) finds the eigenvalues 1.354, 0.946 and 0.778. Two checks by hand. First, the eigenvalues add up to the diagonal of C:

$$1.354 + 0.946 + 0.778 = 3.078$$
$$3 \times 1.026 = 3.078$$

(Before rounding, both sums are 3.077, the total used in Figure 9.)

Second, the **eigenvector** (G-666) $(0.54, 0.66, 0.53)$ for 1.354 satisfies $Cv = \lambda v$, row by row:

Row 1:

$$1.026 \times 0.54 + 0.205 \times 0.66$$

$$+0.080 \times 0.53$$
$$= 0.554 + 0.135 + 0.042 = 0.732$$
$$1.354 \times 0.54 = 0.732$$

Row 2:

$$0.205 \times 0.54 + 1.026 \times 0.66$$
$$+0.198 \times 0.53$$
$$= 0.111 + 0.677 + 0.105 = 0.893$$
$$1.354 \times 0.66 = 0.893$$

Row 3:

$$0.080 \times 0.54 + 0.198 \times 0.66$$
$$+1.026 \times 0.53$$
$$= 0.043 + 0.131 + 0.544 = 0.718$$
$$1.354 \times 0.53 = 0.718$$

Python: Steps 4 and 5.

```
# sort largest eigenvalue first
order = np.argsort(values)[::-1]
values, vectors = values[order], vectors[:, order]
# eigenvectors are the COLUMNS: take the first 2 columns
W = vectors[:, :2].T          # shape (2, 3)
Z = X @ W.T                   # (40, 3) @ (3, 2) = (40, 2)
```

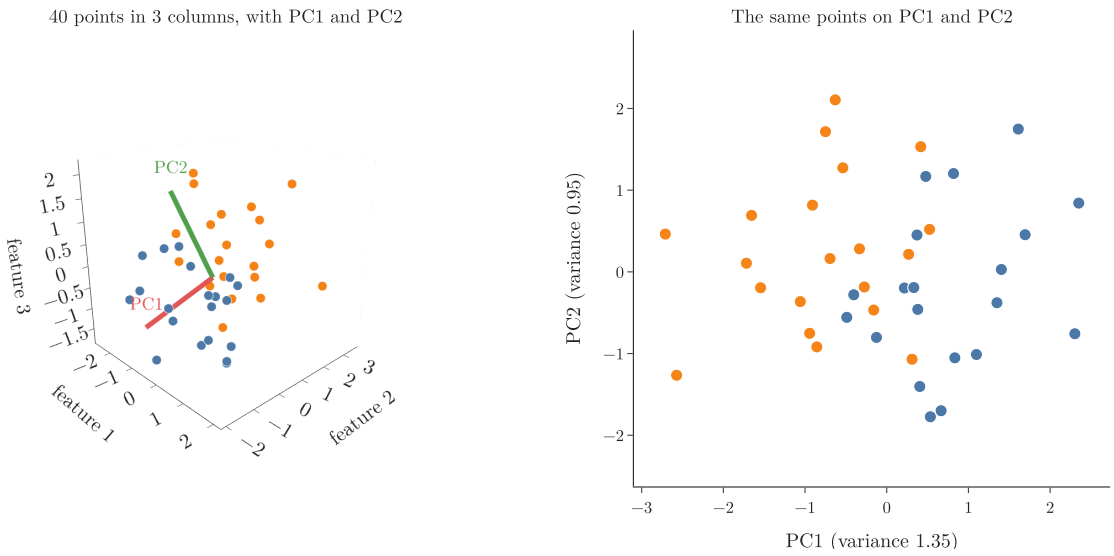


Figure 8: The 40 points in 3D with PC1 and PC2, and after projection

Figure 8 shows the result. On the left, the 3D data with the two principal components drawn through it. On the right, the same 40 points described by just PC1 and PC2.

PC1 holds 44% of the total variance and PC2 31%, so the 2D picture keeps 75% of the spread. The two classes stay mostly apart along PC1.

6.1 Reading the result: the recipe and the scree plot

> **Key point:** An eigenvector is a recipe: how much of each original feature goes into the new one. An **eigenvalue** (G-665) divided by the sum of all eigenvalues is the share of the variance that component keeps.

The recipe. Each new **feature** (G-772) is a weighted sum of the old ones, a **linear combination** (G-1091), and the weights are the entries of the **eigenvector** (G-666). These weights are called **loadings** (G-2204) (or loading scores).

1. **Flats.** PC1 is $(0.707, 0.707)$, so the position of a flat along PC1 is $0.707 \times$ rooms $+$ $0.707 \times$ washrooms (after centring). The recipe uses equal parts of both features: PC1 is a "size of the flat" feature.
2. **The 3-feature example.** PC1 is $(0.54, 0.66, 0.53)$ (NumPy prints it with every sign flipped, which is the same line), so all three features go in with about the same weight. PC2 is $(-0.69, -0.01, 0.72)$: feature 3 minus feature 1, with almost nothing of feature 2.

A feature with a loading near 0 hardly matters for that component; a feature with a large loading (positive or negative) matters a lot.

The scree plot. Each eigenvalue is the variance along its component (Section 4.3), and

together the eigenvalues add up to the total variance. So the share kept by one component is its eigenvalue divided by the sum:

$$\text{share of PC1} = \frac{1.354}{1.354 + 0.946 + 0.778}$$

$$\text{share of PC1} = \frac{1.354}{3.077} = 0.44$$

A bar chart of these shares, one bar per component, is called a **scree plot** (G-2205). Figure 9 builds it for the example: 44, 31 and 25 percent. The orange line adds the bars up: PC1 and PC2 together keep 75 percent, which is the number quoted under Figure 8.

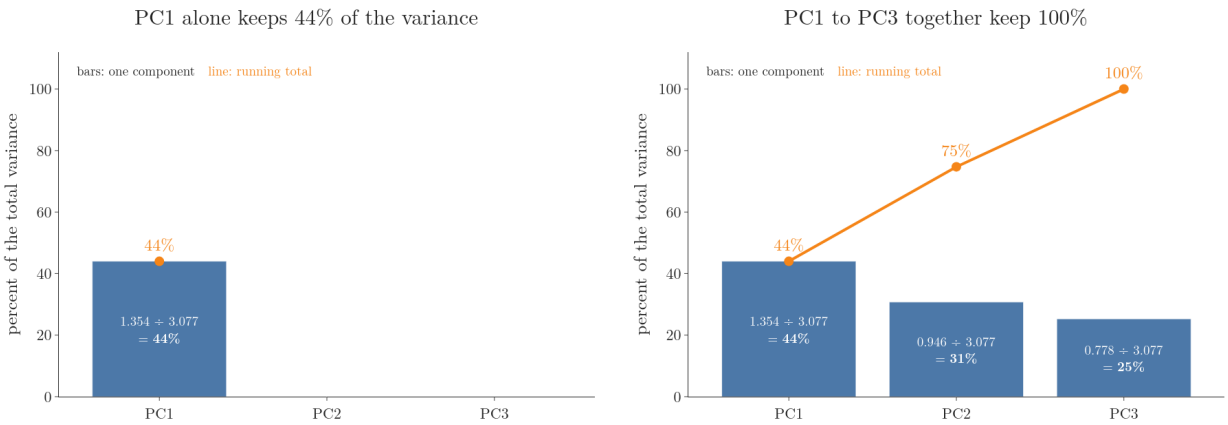


Figure 9: The scree plot of the 3-feature example, built one component at a time. Bars: eigenvalue divided by the sum of eigenvalues. Orange line: the running total.

How many components to keep uses the same shares, there called the explained variance ratio, to choose how many components to keep.

Extra: A frequent bug: `np.linalg.eig` and `np.linalg.eigh` return the eigenvectors as the **columns** of the result, not its rows. Writing `vectors[0:2]` takes two rows, which are not principal components; along them the data's variance is 1.01 and 1.11 instead of 1.35 and 0.95. `eig` also does not sort its output by eigenvalue. Always sort, and take `vectors[:, :k]`.

Python: The same result from scikit-learn.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
Z_sk = pca.fit_transform(X)
pca.explained_variance_        # [1.354, 0.946]: the eigenvalues
pca.explained_variance_ratio_  # [0.44, 0.307]: share of the variance
```

The columns of `Z_sk` equal those of `Z`, possibly with the sign flipped: an eigenvector and its negative lie on the same line.

Extra: `np.cov` and scikit-learn divide by $n-1$, not by n as in the formulas above, which is why the diagonal of C is 1.026 and not 1. Dividing by $n-1$ only scales every eigenvalue by $n/(n-1)$; the eigenvectors stay the same.

7. Summary

Step	What it does	Tool	Why it matters
1. Mean centre	moves the data's centre to the origin	`StandardScaler` (or subtract the mean)	the projection gives centred coordinates only if the data is centred
2. Covariance matrix	spread (diagonal) and orientation (off-diagonal)	`np.cov`	it holds everything PCA needs about the cloud's shape
3. Eigen-decomposition	directions that only stretch, and by how much	`np.linalg.eigh`	the top eigenvector is the direction of greatest spread, found without trying every angle
4. Keep the top k	sort by eigenvalue; the first k are kept	`np.argsort`	the largest eigenvalues hold the most variance
5. Project	new coordinates $Z = XW^{\mathsf{T}}$	`@` (matrix product)	one matrix product turns every row into k new features

- PCA's objective: the **unit vector** (G-2048) u that maximises the variance of the projections $u^{\mathsf{T}}x_i$, because a wide spread of shadows keeps the points apart.
- Covariance shows how two **features** (G-772) move together; its sign gives the direction, so it separates clouds that variance alone cannot (Data A and Data B).
- The **covariance matrix** (G-495): variances on the diagonal, covariances off it, symmetric, so it describes both the spread and the orientation of the data.
- Eigenvectors keep their direction under a matrix; **eigenvalues** (G-665) are their stretch, $Av = \lambda v$.
- The covariance matrix's top **eigenvector** (G-666) is PC1; its eigenvalue is the variance along PC1, so PCA solves its objective by computing the eigenvectors of one matrix, for 3 or 1,000 features alike.
- Eigenvectors are the columns of NumPy's output: sort them and take `vectors[:, :k]`, because taking rows gives directions that are not principal components.
- An eigenvector's entries (**loadings** (G-2204)) are the recipe of the new feature; eigenvalue ÷ sum of eigenvalues is the share of variance it keeps (scree plot), so the shares tell how many components are worth keeping.
- Together these answer the opening question: the direction of largest variance is the top eigenvector of the covariance matrix, and the five steps turn that into new features.

8. Sources

Built from

- CampusX, "Principle Component Analysis (PCA) | Part 2 | Problem Formulation and Step by Step Solution", YouTube, https://www.youtube.com/watch?v=tXXnxjj2wM4
- Sanderson, G. (3Blue1Brown), "Eigenvectors and eigenvalues | Chapter 14, Essence of linear algebra", YouTube, https://www.youtube.com/watch?v=PFDu9oVAE-g (the matrix and picture of section 4.2)
- StatQuest with Josh Starmer, "StatQuest: Principal Component Analysis (PCA), Step-by-Step", YouTube, https://www.youtube.com/watch?v=FgakZw6K1QQ (the recipe reading of an eigenvector, loading scores and the scree plot of section 6.1)

Other references

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, Section 12.1.1, "Maximum variance formulation". Springer.
- Strang, G. (2016). *Introduction to Linear Algebra*, 5th edition, Section 6.4, "Symmetric matrices". Wellesley-Cambridge Press.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Unit vector (G-2048)	A vector of length 1, used to describe a direction.
Objective function (G-1372)	The quantity an algorithm tries to make as large or as small as possible.
Covariance matrix (G-495)	A square table with every feature's variance on the diagonal and every pair's covariance off it, so it sums up how the data spreads and which features move together; PCA takes its eigenvectors.
Eigenvector (G-666)	A non-zero vector that a matrix only stretches or shrinks, without turning it off its own line: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvectors of the covariance matrix are the directions of the principal components.
Identity matrix (G-915)	The matrix that leaves every vector unchanged.
Eigenvalue (G-665)	The number by which a matrix stretches or shrinks one of its eigenvectors: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvalues of the covariance matrix are the variances along the principal components.

Term	Meaning
Eigen-decomposition (G-663)	Finding all the eigenvalues and eigenvectors of a matrix. In PCA it is done on the covariance matrix: the eigenvectors give the directions of the principal components and the eigenvalues the variance along each.
Loading (loading score) (G-2204)	How much one original feature counts in a principal component (its weight there); it is one entry of that component's eigenvector.
Scree plot (G-2205)	A bar chart with one bar per principal component, showing its share of the total variance (eigenvalue ÷ sum of eigenvalues); it shows how many components are worth keeping.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Observation (G-1374)	One record of a dataset: one row of the data table.
Vector (G-2081)	A list of numbers: a 1D tensor.
Dot product (G-634)	Multiplying two vectors entry by entry and adding the products, $u^{\mathsf{T}}x$, which gives one number. It is large when the vectors point the same way, so it measures similarity and gives projections.
Transpose (G-2012)	A matrix or vector with rows and columns swapped; it turns a column vector u into a row u^{T}, so $u^{\mathsf{T}}x$ is the dot product.
Linear transformation (G-1097)	A change of the whole plane by a matrix that keeps grid lines straight and evenly spaced, and the origin fixed; algebraically, $L(\mathbf{v}+\mathbf{w}) = L(\mathbf{v}) + L(\mathbf{w})$ and $L(c\mathbf{v}) = cL(\mathbf{v})$.
Mean centring (G-1195)	Subtracting the mean from every value, so the column's mean becomes 0.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Linear combination (G-1091)	A sum of vectors each multiplied by a number, $a_1\mathbf{v}_1 + \cdots + a_k\mathbf{v}_k$; changing the numbers reaches different vectors, and all the vectors it can reach form the span.
Covariance (G-496)	A number that says how two numerical features move together: positive if they rise together, negative if one rises as the other falls. Its size depends on the units.

Term	Meaning
Explained variance (G-728)	How much of the data's spread one principal component keeps: the variance of the data along that component, equal to its eigenvalue.